

E0 230: Assignment 2

Sayak Das
SR No: 24235
MTech AI

October 20, 2024

I have reused code from Assignment 1.

Question 1 : Conjugate Gradient Descent

1. A is an $n \times n$ symmetric, PD matrix.

Yes, the equation $Ax = b$ has a unique solution because the number of equations is same as number of unknowns in the linear system defined by $Ax = b$.

Let the quadratic minimization problem be defined as,

$$\min_{x \in \mathbb{R}^d} \frac{1}{2} x^T A x - b^T x$$

Because A is PD, the gradient value at minimum, x^* , must be 0. Therefore,

$$\begin{aligned} \nabla f(x^*) &= Ax^* - b = 0 \\ \mathbf{Ax}^* &= \mathbf{b} \end{aligned}$$

2. A is an $n \times n$ symmetric, PD matrix. Run conjugate gradient descent algorithm.

We use the initial starting point $x^{(0)} = [0, 0, 0, 0, 0]$. We observe that the algorithm converges in 5 iterations. With $x^* = [2, 5, 6, 2, 1]$. The updates are given in Table 1.

3. A is an $m \times n$ matrix with $m > n$.

The equation $Ax = b$ does not have a unique solution. Because number of equations is more than number of unknowns in the linear system defined by $Ax = b$. Hence, the system is linearly dependent and there will be more than one solution, which implies that the solution is not unique.

Iterations	$\mathbf{x}$
1	[2.72, 3.92, 4.75, 2.61, 2.53]
2	[2.18, 5.13, 5.49, 2.44, 0.99]
3	[1.99, 5.03, 5.97, 1.97, 1.01]
4	[2.00, 5.00, 5.99, 1.99, 0.99]
5	[2, 5, 6, 2, 1]

Table 1: Value of x at each iteration with conjugate gradient descent.

Iterations	$\mathbf{x}$
1	$[2, 5, -2.22 \times 10^{-16}, 0, 9]$
2	$[2, 5, 3.83 \times 10^{-16}, -3.13 \times 10^{-16}, 9]$
3	$[2, 5, 3.53 \times 10^{-16}, -2.98 \times 10^{-16}, 9]$

Table 2: Value of x at each iteration with conjugate gradient descent with minimizing error term.

Let's try to minimize the square of the error term. Because $\|Ax - b\| \geq 0$, minimizing $\|Ax - b\|$ is same as minimizing $\|Ax - b\|^2$.

$$\min_{x \in \mathbb{R}^d} \|Ax - b\|^2$$

We can now express the error term as an inner product,

$$\begin{aligned} \|Ax - b\|^2 &= (Ax - b)^T (Ax - b) \\ &= x^T A^T Ax - b^T Ax - b^T Ax + b^T b \\ &= x^T A^T Ax - 2b^T Ax + b^T b \end{aligned}$$

So, the new quadratic minimization problem becomes,

$$\min_{x \in \mathbb{R}^d} x^T A^T Ax - 2b^T Ax + b^T b$$

This quadratic function has gradient value 0 at minimum x^* , so,

$$\begin{aligned} \nabla f(x^*) &= 2A^T Ax^* - 2b^T A = 0 \\ 2A^T Ax^* &= 2b^T A \\ x^* &= (A^T A)^{-1} b^T A \end{aligned}$$

Hence, the minimization problem has a unique solution when $A^T A$ is invertible.

4. A is an $m \times n$ matrix with $m > n$. Run conjugate gradient descent algorithm to minimize the error term.

We use the initial starting point $x^{(0)} = [0, 0, 0, 0, 0]$. We observe that the algorithm converges in 3 iterations. With $x^* = [2, 5, 3.53 \times 10^{-16}, -2.98 \times 10^{-16}, 9]$. The updates are given in Table 2.

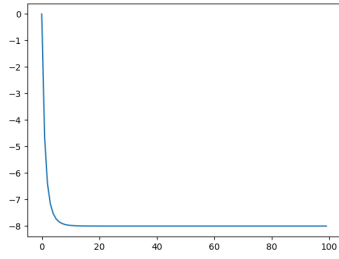
Question 2 : Newton's Method

1. Gradient Descent for 100 iterations with five different step-sizes.

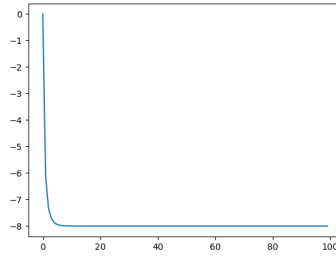
We have used step-sizes (α) 0.1, 0.15, 0.2, 0.25 and 0.3 for this experiment. The plots of function values for each of these experiments are presented in Figure 1. For all the step-sizes, $x^{(100)} = [1, 1, 1, 1, 1]^T$.

2. Newton for 100 iterations.

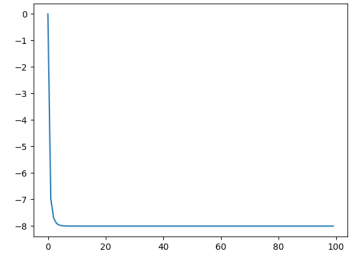
We have run Newton for 100 iterations starting at $x_0 = [0, 0, 0, 0, 0]^T$. The algorithm yields $x^* = [1, 1, 1, 1, 1]^T$. Which is exactly the same as gradient descent for all step-sizes. Although the algorithm need not be run for 100 iteration, a quick check reveals that Newton converges to $x^* = [1, 1, 1, 1, 1]^T$ in just one step. Function value plot for both is presented in Figure 2.



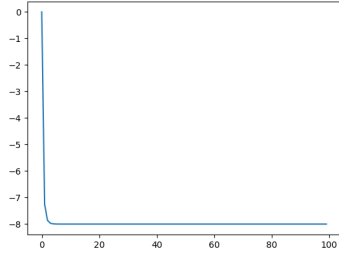
(a) $\alpha = 0.1$



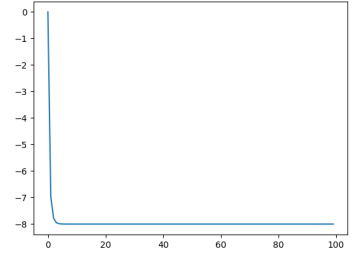
(b) $\alpha = 0.15$



(c) $\alpha = 0.2$



(d) $\alpha = 0.25$

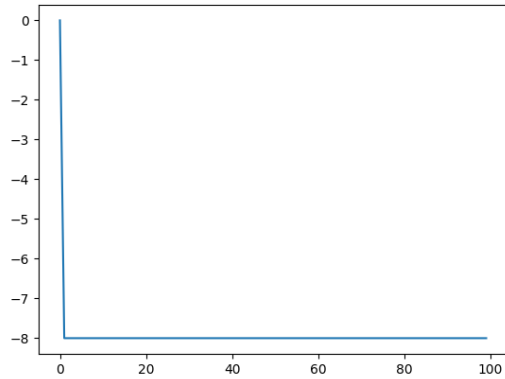


(e) $\alpha = 0.3$

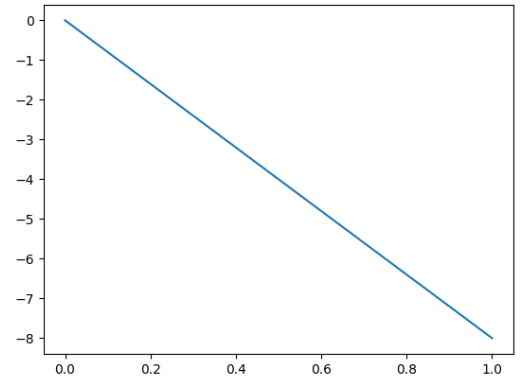
Figure 1: Gradient Descent with different step-sizes: X-axis is number of iterations, Y-axis is function value.

Step-size	$\mathbf{x}^*$
0.1	[1, 1, 1, 1, 1]
0.15	[1, 1, 1, 1, 1]
0.2	[1, 1, 1, 1, 1]
0.25	[1, 1, 1, 0, 1]
0.3	[1, 1, 1, -4×10^{14} , 1]

Table 3: Value of $\mathbf{x}^*$ for each step-size. All the Gradient Descent algorithms were run for 100 iterations.

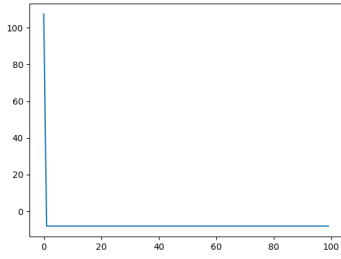


(a) Number of iterations = 100

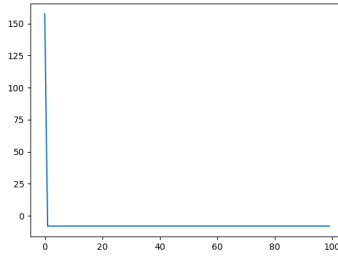


(b) Number of iterations = 1

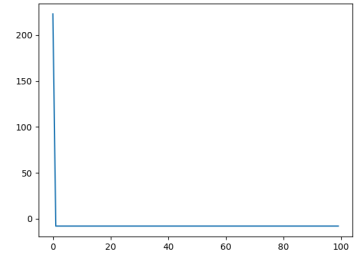
Figure 2: Newton with different iterations: X-axis is number of iterations, Y-axis is function value.



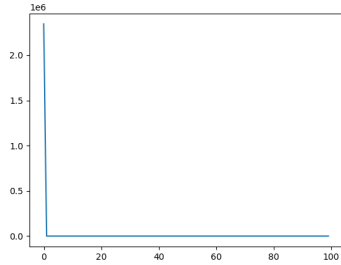
(a) $x^{(0)} = [5, 1, 2, 9, 0]$



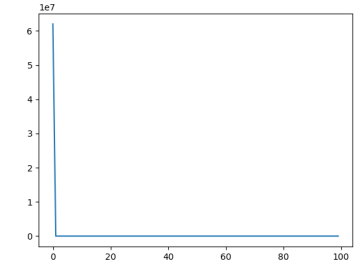
(b) $x^{(0)} = [-1, -2, -3, -4, -5]$



(c) $x^{(0)} = [1, 0, 6, 0, -8]$



(d) $x^{(0)} = [200, 683, 169, -947, -1]$



(e) $x^{(0)} = [1900, -1683, -7169, -947, 45]$

Figure 3: **Newton** with five different starting points, $x^{(0)}$: X-axis is number of iterations, Y-axis is function value..

3. Newton starting with five different initial points.

The initial points and function value at each iteration for each is presented in Figure 3. For all starting points, **Newton** converges to $x^* = [1, 1, 1, 1, 1]^T$ in one step. Notice how for all of our experiments the minima stayed the same. Convergence of **Newton** in one step and uniqueness of minima implies that the underlying function, **f2**, is quadratic and it's Hessian is Positive Definite. Definiteness of the Hessian also implies the function is strongly convex. So, we claim that **Newton** converges in one step for strongly convex functions. We prove this fact in the following section.

4. Newton converges in one step for strongly convex functions.

Let the minimization problem be defined as the following,

$$\min_{x \in \mathbb{R}^d} f(x) = \frac{1}{2}x^T A x - b^T x \quad \text{where } b \in \mathbb{R}^d \text{ and } A \succ 0$$

Positive definiteness of Hessian A ensures that the function is strongly convex. So, the minima, x^* , is unique. We know at x^* the gradient will be 0. Therefore,

$$\begin{aligned} \nabla f(x^*) &= A x^* - b = 0 \\ x^* &= A^{-1} b \quad [\text{Because } A \text{ is symmetric PD, } A^{-1} \text{ exists.}] \end{aligned}$$

We employ the such strict rule of strong convexity for a reason. Suppose the Hessian was positive semi-definite. Then one of it's eigenvalues would have been zero. Hence $\det(A) = 0$. So the inverse would not exist.

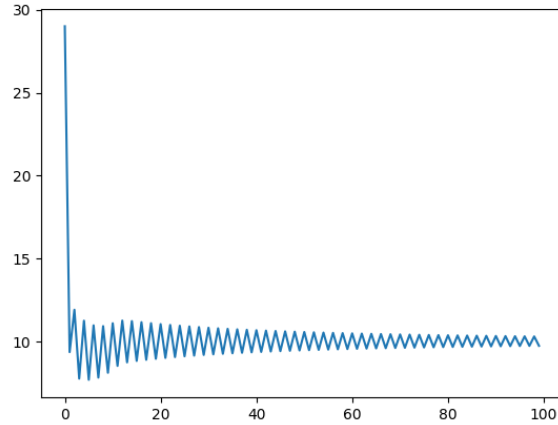


Figure 4: Gradient Descent with step-size 0.1 for 100 iterations: X-axis is number of iterations, Y-axis is function value.

We show that for any starting point, **Newton** reaches the minima in one iteration. **Newton** update is given as,

$$x^{(k+1)} = x^{(k)} - (H(x^{(k)})^{-1} \nabla f(x^{(k)})) \quad \text{where } H(x^{(k)}) \text{ is the Hessian at } x^{(k)}$$

Let $x^{(0)} \in \mathbb{R}^d$ be any starting point. Then the **Newton** update is given as,

$$\begin{aligned} x^{(1)} &= x^{(0)} - (H(x^{(0)})^{-1} \nabla f(x^{(0)})) \\ &= x^{(0)} - A^{-1}(Ax^{(0)} - b) \\ &= x^{(0)} - A^{-1}Ax^{(0)} + A^{-1}b \\ &= x^{(0)} - I_d x^{(0)} + A^{-1}b \\ &= x^{(0)} - x^{(0)} + A^{-1}b \\ &= A^{-1}b \end{aligned}$$

We have just proved that for strongly convex functions, regardless of the starting point, **Newton** converges in one step.

Question 3 : Newton's Method continued

1. Gradient Descent with step-size 0.1 for 100 iterations.

The best function value obtained over all the 100 iterations was $f(x^*) = 7.68$ with $x^* = [-0.47, 2.17 \times 10^{-17}, 0.17, -0.39, 0.17]$. The plot is presented in Figure 4. Notice the oscillation in function value.

2. Reason for oscillations in Gradient Descent.

One possible reason for the oscillation can be the step-size is greater than the interval it is trying to minimize. Hence the function value goes back-and-forth, unable to reach the minima. In fact, using Gradient Descent with a decreasing step-size (from Assignment 1) overcomes the problem. We present the plot for Gradient Descent with decreasing step-size in Figure 5. We also achieve a better function value

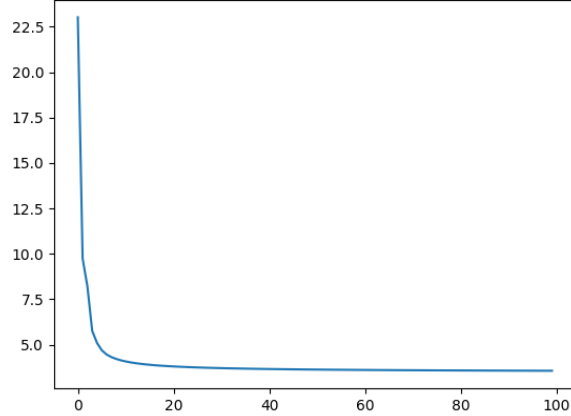


Figure 5: Gradient Descent with decreasing step-size ($\alpha_0 = 0.1$) for 100 iterations: X-axis is number of iterations, Y-axis is function value.

at the end of 100 iterations, $f(x^{(100)}) = 3.56$ with $x^{(100)} = [-5.42 \times 10^{-17}, 1.19 \times 10^{-3}, 2.15 \times 10^{-1}, 2.87 \times 10^{-2}, 9.06 \times 10^{-6}]$.

3. Newton's Method did not converge.

No, the algorithm did not converge. In fact, the gradient at each iteration started increasing extremely fast and the algorithm gave $x^* = \text{nan}$ value at the end of 10 (therefore also after 100) iterations. Possible reason for this can be that, the function is not convex everywhere. Which means the Hessian at those points is not positive definite. So the inverse of the Hessian produces very large values forcing the algorithm to take large steps and diverge. We also said that the gradient at each iteration was increasing very fast. Which means the slope of the function is very steep at those points. So **Newton** diverges. We can use **Newton** once we reach the Newton region using Gradient Descent. This is exactly what we do in the next question.

4. Newton's Game

We have ran Gradient Descent with a step-size (α) 0.01. We observe the least cost when we ran Gradient Descent for 97 iterations followed by Newton Method for the last 3 iterations. Hence the total cost of the procedure is $(97 * 1 + 3 * 25) = 172$. The function value at the end of the procedure is $x^{(100)} = 3.46$. The plot is presented in Figure 6.

Question 4 : Quasi-Newton Methods

1. Derive an update expression for approximating the Hessian with a scalar multiple of the identity matrix

At each step $k + 1$, we have $x^{(k+1)}$, $x^{(k)}$, $\nabla f(x^{(k+1)})$ and $\nabla f(x^{(k)})$.

Let $\delta^{(k)} = x^{(k+1)} - x^{(k)}$ and $\gamma^{(k)} = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)})$. We want to approximate Hessian at each step. Let $H^{(k)} \approx \sigma^{(k)} \mathbf{I}$. We want an update expression for $\sigma^{(k)}$.

Now at each step the update must follow the Quasi-Newton condition,

$$H^{(k+1)} \gamma^{(k)} = \delta^{(k)}$$

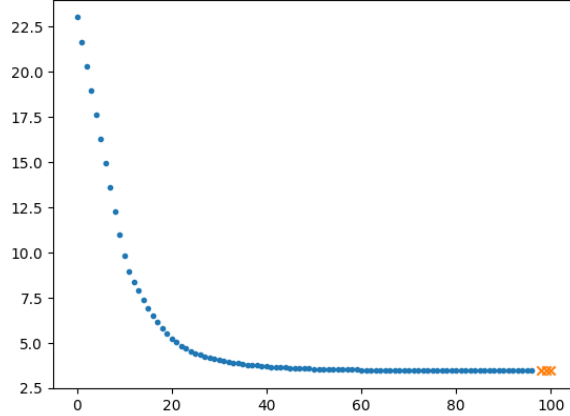


Figure 6: Gradient Descent with step-size (α_0) 0.1 for 97 iterations (blue dots) followed by 3 Newton iterations (red crosses): X-axis is number of iterations, Y-axis is function value.

Now we replace $H^{(k+1)}$ with $\sigma^{(k+1)}\mathbf{I}$ in the above equation,

$$\begin{aligned} H^{(k+1)}\gamma^{(k)} &= \delta^{(k)} \\ \sigma^{(k+1)}\mathbf{I}^{(k)}\gamma^{(k)} &= \delta^{(k)} \\ \sigma^{(k+1)}\gamma^{(k)} &= \delta^{(k)} \end{aligned}$$

Now we can multiply both sides by either $\gamma^{(k)}$ or $\delta^{(k)}$. Giving us the result,

$$\sigma^{(k+1)} = \frac{\delta^{(k)T}\gamma^{(k)}}{\gamma^{(k)T}\gamma^{(k)}} = \frac{\delta^{(k)T}\delta^{(k)}}{\gamma^{(k)T}\delta^{(k)}}$$

Hence, the we can approximate the Hessian with either one of these following expressions,

$$H^{(k+1)} \approx \frac{\delta^{(k)T}\gamma^{(k)}}{\gamma^{(k)T}\gamma^{(k)}}\mathbf{I} = \frac{\delta^{(k)T}\delta^{(k)}}{\gamma^{(k)T}\delta^{(k)}}\mathbf{I}$$

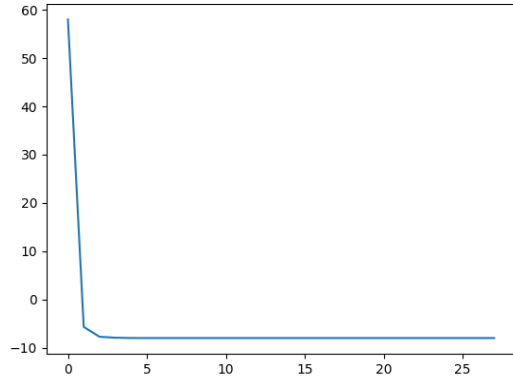
We have used only one extra variable, σ , apart from what was given to us. Hence, the memory constraints are also satisfied.

2. Comparing quasi-Newton rank 1 update with quasi-Newton Identity approximation and Gradient Descent with different step-sizes.

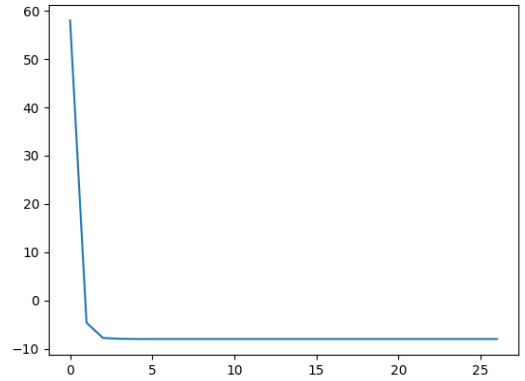
We have already experimented with Gradient Descent with different step sizes on **f2** in Question 2. The figures and table containing the results can be seen in Figure 1 and Table 3 respectively.

When approximating the Hessian with a scalar multiple of the Identity, be it any of the two equations derived previously, both converge to $x^* = [1, 1, 1, 1, 1]$. It can be verified that this is same as the first three experiments with Gradient Descent (*see Table 3*). We present the graphs of this experiment in Figure 7.

When using Quasi-Newton rank 1 update we see that the algorithm also converges to $x^* = [1, 1, 1, 1, 1]$. The algorithm also converges in only 5 iterations. The plot of the algorithm is presented in Figure 8.



$$(a) \ H^{(k+1)} \approx \frac{\delta^{(k)T} \gamma^{(k)}}{\gamma^{(k)T} \gamma^{(k)}} \mathbf{I}$$



$$(b) \ H^{(k+1)} \approx \frac{\delta^{(k)T} \delta^{(k)}}{\gamma^{(k)T} \delta^{(k)}} \mathbf{I}$$

Figure 7: Quasi-Newton with scalar identity approximation: X-axis is number of iterations, Y-axis is function value.

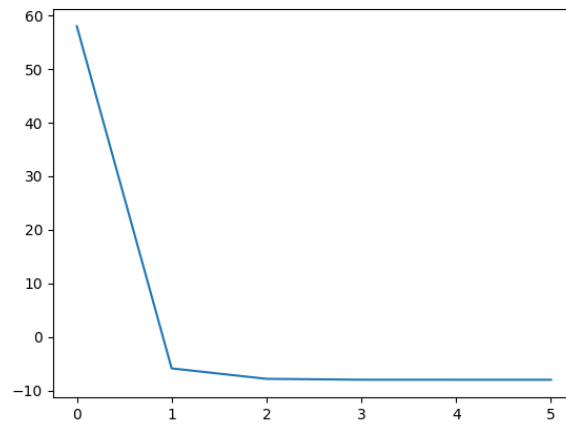


Figure 8: Quasi-Newton rank 1 update: X-axis is number of iterations, Y-axis is function value.